

La solution : les outils de fonction personnalisés

Écrire des fonctions Python, obtenir des capacités agentiques

Référence : [Documentation ADK - Outils de fonction](#)

Extrait de la documentation ADK :

"Transformer une fonction Python en un outil est un moyen simple d'intégrer une logique personnalisée à vos agents." Lorsque vous attribuez une fonction à la liste `tools` d'un agent, le framework l'encapsule automatiquement en tant que `FunctionTool`.

Fonctionnement :

1. **Vous écrivez** une fonction Python standard avec votre logique métier.
2. **ADK inspecte** le nom de la fonction, la docstring, les paramètres et les indications de type.
3. **ADK génère** un schéma décrivant l'outil pour le LLM.
4. **L'agent utilise** l'outil basé sur un schéma et une docstring.

Exemple simple :

```
from google.adk.agents import LlmAgent

# Étape 1 : Écrire une fonction Python
def calculate_shipping_cost(weight_kg: float, country: str) -> dict:
    """Calcule les frais de port en fonction du poids du colis et de sa destination."""
    rates = {"usa": 10, "canada": 12, "uk": 15}

    if country.lower() not in rates:
        return {"status": "error", "error_message": f"Nous ne livrons pas dans le pays suivant : {country}"}

    cost = weight_kg * rates[country.lower()]
    return {"status": "success", "cost_usd": cost}

# Étape 2 : Ajouter la fonction à la liste d'outils de l'agent
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Tu aides les clients à estimer les frais de port.",
    tools=[calculate_shipping_cost] # ADK encapsule automatiquement en tant que FunctionTool
)
```

Fonctionnalités proposées :

-  **Intégrer votre logique métier** directement dans les capacités agentiques
-  **Accéder à vos systèmes propriétaires** avec du code Python
-  **Implémenter des calculs spécifiques à un domaine** qui sont propres à votre

- entreprise
-  **Contrôler totalement** l'implémentation et le comportement
-  **Générer automatiquement des schémas** à partir de la signature de votre fonction

Comment les outils personnalisés s'intègrent-ils aux agents ?

```
sequenceDiagram
    participant User
    participant Agent
    participant Tool as Custom Tool
    participant System as Your System/API

    User->>Agent : "Calcule les frais de port vers le Canada."
    Agent->>Agent : analyse la requête et sélectionne l'outil
    Agent->>Tool : calculate_shipping_cost(weight=5, country="Canada")
    Tool->>System : consulte les tarifs dans la base de données
    System-->>Tool : informations tarifaires
    Tool->>Tool : calcule 5 kg × 12 $/kg
    Tool-->>Agent : {"status": "success", "cost_usd": 60}
    Agent-->>User : "Les frais de port sont de 60 USD."
```

Note over Agent,Tool : le LLM décide QUAND appeler
L'outil exécute CE QU'il faut faire

Les outils personnalisés permettent aux agents d'accéder à votre logique métier et à vos systèmes.

Concepts fondamentaux

1. L'importance des signatures de fonction

Référence : [Documentation ADK - Outils de fonction](#)

Extrait de la documentation ADK :

"La signature de la fonction (y compris les types de paramètres et le type renvoyé) est cruciale. ADK utilise ces informations pour générer le schéma que le LLM voit."

Éléments critiques :

1. Nom de la fonction (descriptif)

Le LLM utilise le nom de la fonction pour comprendre ce que fait l'outil.

```
#  Bonne pratique : nom descriptif, structure verbe-nom
def get_shipping_cost(weight: float, destination: str) -> dict:
    """Récupère les frais de port d'un colis."""
    pass

def calculate_loyalty_points(purchase_amount: float) -> dict:
```

```

    """Calcule les points de fidélité gagnés lors d'un achat."""
    pass

def search_available_flights(destination: str, date: str) -> dict:
    """Recherche les vols disponibles vers une destination."""
    pass

# ❌ Mauvaise pratique : noms génériques et peu clairs
def process(data: float) -> dict: # Traiter quoi ?
    pass

def do_stuff(x: str) -> dict: # Quel truc ?
    pass

def handler(amount: float) -> dict: # Gérer quoi ?
    pass

```

2. Indications de type (obligatoires)

Les indications de type signalent à ADK les types que le LLM doit fournir.

```

# ✅ Bonne pratique : indications de type pour tous les paramètres et toutes les valeurs de retour
def lookup_order(order_id: str, user_id: int) -> dict:
    """Consulte les informations sur les commandes."""
    pass

def calculate_discount(price: float, customer_tier: str) -> dict:
    """Calcule la remise en fonction du niveau du client."""
    pass

# ❌ Mauvaise pratique : aucune indication de type ; le LLM ne sait pas quels types fournir
def lookup_order(order_id, user_id): # De quels types s'agit-il ?
    pass

def calculate_discount(price, customer_tier): # Types inconnus
    pass

```

3. Types de paramètres

Utilisez des types sérialisables en JSON que les LLM comprennent :

- ✅ **Compatibles** : `str`, `int`, `float`, `bool`, `list`, `dict`
- ❌ **À éviter** : classes personnalisées complexes, objets, descripteurs de fichiers

```

# ✅ Bonne pratique : types simples et sérialisables en JSON
def book_flight(
    destination: str,
    departure_date: str,
    passengers: int
) -> dict:
    pass

# ❌ Mauvaise pratique : types personnalisés complexes
from datetime import datetime

```

```

from custom_models import Customer

def book_flight(
    destination: str,
    departure_date: datetime, # Non sérialisable en JSON
    customer: Customer # Classe personnalisée
) -> dict:
    pass

```

4. Paramètres obligatoires et facultatifs

Extrait de la documentation ADK :

"Ne définissez pas de valeurs par défaut pour les paramètres. Les valeurs par défaut ne sont pas prises en charge ni utilisées de manière fiable par les modèles sous-jacents."

Recommandation : utilisez tous les paramètres requis.

```

# ✓ Pratique recommandée : tous les paramètres sont requis
def book_flight(destination: str, date: str, passengers: int) -> dict:
    """Réserve un vol."""
    pass

# ⚠ Pratique non recommandée : il se peut que les valeurs par défaut ne fonctionnent
pas correctement
def search_flights(
    destination: str,
    max_price: float = 1000.0, # La valeur par défaut peut être ignorée
    class_type: str = "economy" # La valeur par défaut peut être ignorée
) -> dict:
    """Recherche des vols."""
    pass

```

2. Les docstrings sont essentielles

Référence : [Documentation ADK - Définir des fonctions d'outil efficaces](#)

Extrait de la documentation ADK :

"La docstring de votre fonction sert de description à l'outil et est envoyée au LLM. Par conséquent, une docstring bien rédigée et complète est essentielle pour que le LLM comprenne comment utiliser l'outil efficacement."

Modèle de docstring :

```

def tool_name(param1: type1, param2: type2) -> dict:
    """[Résumé en une ligne de ce que fait cet outil]

    [Facultatif : contexte supplémentaire pour indiquer quand utiliser cet outil]

```

```

Args:
    param1 (type1) : [Description de param1]
    param2 (type2) : [Description de param2]

Renvoie :
    dict : [Description de la structure de retour]
        En cas de succès : {'status': 'success', 'key': value}
        En cas d'erreur : {'status': 'error', 'error_message': 'explanation'}
"""
# Implémentation
pass

```

Exemple complet :

```

def calculate_shipping_cost(weight_kg: float, country: str) -> dict:
    """Calcule les frais de port en fonction du poids du colis et du pays de
    destination.

    Utilise cet outil lorsqu'un client s'enquiert des frais de port de sa commande.

    Args:
        weight_kg (float) : le poids du colis en kilogrammes.
        country (str) : le nom du pays de destination.

    Renvoie :
        dict : informations sur les frais de port.
            En cas de succès : {'status': 'success', 'cost_usd': 25.50,
'delivery_days': 5}
            En cas d'erreur : {'status': 'error', 'error_message': 'Pays non
disponible'}
    """
    shipping_rates = {
        "usa": {"rate_per_kg": 10, "days": 3},
        "canada": {"rate_per_kg": 12, "days": 5},
        "uk": {"rate_per_kg": 15, "days": 7},
    }

    country_key = country.lower()
    if country_key not in shipping_rates:
        return {
            "status": "error",
            "error_message": f"La livraison n'est actuellement pas disponible vers le
pays suivant : {country}."
        }

    rate_info = shipping_rates[country_key]
    cost = weight_kg * rate_info["rate_per_kg"]

    return {
        "status": "success",
        "cost_usd": round(cost, 2),
        "delivery_days": rate_info["days"],
        "destination": country
    }

```

Ce que voit le LLM :

Tool: calculate_shipping_cost

Description :

Calcule les frais de port en fonction du poids du colis et du pays de destination.
Utilise cet outil lorsqu'un client s'enquiert des frais de port de sa commande.

Paramètres :

- weight_kg (float) : le poids du colis en kilogrammes
- country (str) : le nom du pays de destination

Renvoie :

Dictionnaire contenant les informations sur l'état et la livraison

Le LLM utilise ceci pour décider :

- **Quand appeler** : "Le client demande des informations sur les frais de port"
- **Quels paramètres** : weight_kg (float), country (str)
- **Résultat attendu** : dictionnaire contenant cost_usd et delivery_days

Comment ADK convertit votre fonction en schéma LLM :

```
graph TB
    A[Votre fonction Python] --> B[Nom de la fonction]
    A --> C[Indications de type]
    A --> D[Docstring]
    A --> E[Type renvoyé]

    B --> F[Génération de schémas LLM]
    C --> F
    D --> F
    E --> F

    F --> G[Nom de l'outil]
    F --> H[Description de l'outil]
    F --> I[Types de paramètres]
    F --> J[Résultat attendu]

    G --> K[Sélection d'outils de LLM]
    H --> K
    I --> K
    J --> K

    K --> L[L'agent appelle l'outil<br/>avec les paramètres corrects]

    style F fill:#ffeb99
    style K fill:#e1f5ff
```

*ADK génère automatiquement le schéma LLM à partir des métadonnées de votre fonction.
Aucune configuration manuelle n'est requise.*

3. Renvoyer des dictionnaires avec état

Référence : [Documentation ADK - Outils de fonction](#)

Extrait de la documentation ADK :

"Le type renvoyé de prédilection pour un outil de fonction est un dictionnaire en Python... Il est vivement recommandé d'inclure une clé `status` (par exemple, `success`, `error`, `pending`) afin d'indiquer clairement le résultat de l'exécution de l'outil pour le modèle."

Format :

```
# Cas de réussite
return {
    "status": "success",
    "data_key": value,
    "another_key": another_value
}

# Cas d'erreur
return {
    "status": "error",
    "error_message": "explication lisible par l'humain du problème"
}
```

Pourquoi c'est important :

- **Compréhension du LLM** : un état clair permet au LLM de savoir si l'outil a fonctionné.
- **Prochaine action** : le LLM choisit que faire en fonction de l'état (continuer, réessayer, s'excuser).
- **Gestion des exceptions** : les messages d'erreur guident la réponse du LLM à l'utilisateur.
- **Données structurées** : avec un format cohérent, les réponses du LLM sont plus fiables.

Exemple :

```
def lookup_product(product_id: str) -> dict:
    """Recherche les informations produit par ID.

    Args:
        product_id (str) : l'ID du produit à rechercher.

    Renvoie :
        dict : informations produit.
            En cas de succès : {'status': 'success', 'product': {...}}
            En cas d'erreur : {'status': 'error', 'error_message': 'explanation'}
    """

    # Base de données simulée
    products = {
        "PROD001": {"name": "Widget Pro", "price": 99.99, "in_stock": True},
```

```

        "PROD002": {"name": "Gadget Plus", "price": 149.99, "in_stock": False}
    }

    # Cas de réussite
    if product_id in products:
        return {
            "status": "success",
            "product_id": product_id,
            "product": products[product_id]
        }

    # Cas d'erreur
    return {
        "status": "error",
        "error_message": f"Produit {product_id} introuvable dans la base de données."
    }

```

Comment le LLM s'en sert :

Utilisateur : "J'aimerais en savoir plus sur le produit PROD999."

L'agent appelle : `lookup_product("PROD999")`

L'outil renvoie : `{"status": "error", "error_message": "Le produit PROD999 est introuvable dans la base de données."}`

Le LLM voit `status="error"` et répond :

"Je m'excuse, mais je n'ai pas trouvé le produit PROD999 dans notre système. Pourriez-vous vérifier l'ID produit ?"

Format de clé :

```

def your_tool(params) -> dict:
    """Votre docstring."""

    # Valider les entrées
    if validation_fails:
        return {
            "status": "error",
            "error_message": "Explication claire pour l'utilisateur"
        }

    # Effectuer l'opération
    try:
        result = do_something()
        return {
            "status": "success",
            "result_key": result,
            "additional_info": other_data
        }
    except Exception as e:
        return {
            "status": "error",
            "error_message": f"L'opération a échoué : {str(e)}"
        }

```

```
}
```

4. Utiliser l'état dans les outils personnalisés (aperçu)

Lien avec la leçon 4 : Gestion de l'état et de la mémoire

Les outils personnalisés peuvent accéder à l'état de la session et le modifier afin de fournir un comportement personnalisé et contextuel. Le fonctionnement de `tool_context` sera abordé dans les leçons suivantes, mais voici ce qu'il est possible de faire :

Exemple : Outil qui utilise l'état pour la personnalisation

```
def recommend_products(category: str, ctx) -> dict:
    """Recommande des produits en fonction des préférences utilisateur à partir de
    l'état.
```

```
    Args:
        category : catégorie de produits à rechercher
        ctx : contexte de l'outil (donne accès à session.state)
```

```
    Renvoie :
        dict : produits recommandés avec informations de personnalisation
```

```
    Remarque : Le paramètre ctx (tool_context) est traité dans la leçon 6.
    """
```

```
    # Accéder aux préférences utilisateur à partir de l'état
    user_tier = ctx.session.state.get('user:tier', 'standard')
    user_preferences = ctx.session.state.get('user:preferences', {})

    # Personnaliser les recommandations en fonction du niveau et des préférences
    if user_tier == 'premium':
        products = get_premium_products(category, user_preferences)
        discount = 0.15
    else:
        products = get_standard_products(category)
        discount = 0.05
```

```
    return {
        "status": "success",
        "products": products,
        "tier": user_tier,
        "discount_applied": discount
    }
```

Quand utiliser l'état dans les outils :

Personnalisation basée sur les préférences utilisateur

```
# Accéder à user:language pour fournir des résultats localisés
language = ctx.session.state.get('user:language', 'en')
```

✓ Contexte des appels d'outils précédents

```
# Vérifier si l'opération précédente est terminée
prev_status = ctx.session.state.get('temp:last_operation_status')
```

✓ Workflows en plusieurs étapes nécessitant des données intermédiaires

```
# Enregistrer les résultats intermédiaires pour l'outil suivant
ctx.session.state['temp:calculation_result'] = computed_value
```

✓ Configuration spécifique à l'utilisateur

```
# Utiliser les unités préférées de l'utilisateur
units = ctx.session.state.get('user:preferred_units', 'imperial')
```

Espaces de noms d'état dans les outils (de la leçon 4) :

- **temp** : – Données temporaires, supprimées après le tour
- Pas de préfixe – Données de session, supprimées après la fin de la conversation
- **user** : – Préférences utilisateur, conservées d'une session à l'autre
- **app** : – Configuration globale, partagée par tous les utilisateurs

Exemple : Workflow en plusieurs étapes avec état

```
def start_order_process(product_id: str, ctx) -> dict:
    """Lance le processus de commande et l'enregistre dans l'état."""
    # Enregistrer les données intermédiaires dans l'état de session
    ctx.session.state['temp:current_order'] = {
        'product_id': product_id,
        'step': 'initiated',
        'timestamp': time.time()
    }
    return {"status": "success", "message": "Commande commencée"}

def add_shipping_address(address: dict, ctx) -> dict:
    """Ajoute l'adresse de livraison à la commande en cours."""
    # Récupérer les données intermédiaires de l'état
    order_data = ctx.session.state.get('temp:current_order')
    if not order_data:
        return {"status": "error", "error_message": "Aucune commande active"}

    # Mettre à jour la commande dans l'état
    order_data['shipping_address'] = address
    order_data['step'] = 'address_added'
    ctx.session.state['temp:current_order'] = order_data

    return {"status": "success", "message": "Adresse ajoutée"}

def finalize_order(ctx) -> dict:
    """Finalise la commande en utilisant les données de l'état."""
    # Récupérer les données complètes de la commande
    order_data = ctx.session.state.get('temp:current_order')
```

```

if not order_data or order_data['step'] != 'address_added':
    return {"status": "error", "error_message": "Commande non prête"}

# Ordre de traitement (appel d'API, base de données, etc.)
order_id = process_order_in_system(order_data)

# Nettoyer l'état temporaire
del ctx.session.state['temp:current_order']

return {"status": "success", "order_id": order_id}

```

Ce que cela démontre :

- Les outils peuvent consulter les préférences utilisateur à partir de l'espace de noms `user:.`
- Les outils peuvent enregistrer/récupérer des données intermédiaires avec l'espace de noms `temp:.`
- Plusieurs outils se coordonnent par le biais d'un état partagé.
- L'état permet des workflows complexes avec état.

Aperçu : L'intégration complète des outils avec la gestion d'état, y compris le paramètre `ctx` et `tool_context`, est abordée dans les prochaines leçons consacrées aux rappels et aux garde-fous de sécurité.

5. Plusieurs outils fonctionnant ensemble

Modèle : lister plusieurs fonctions dans le paramètre "tools"

Les agents peuvent utiliser plusieurs outils pour accomplir des tâches complexes :

```

from google.adk.agents import LlmAgent

def check_inventory(product_id: str) -> dict:
    """Vérifie si un produit est en stock."""
    # Implémentation
    return {"status": "success", "in_stock": True, "quantity": 5}

def get_product_price(product_id: str) -> dict:
    """Obtient le prix actuel d'un produit."""
    # Implémentation
    return {"status": "success", "price_usd": 99.99}

def calculate_shipping(weight_kg: float, country: str) -> dict:
    """Calcule les frais de port."""
    # Implémentation
    return {"status": "success", "cost_usd": 25.50}

def calculate_total(product_price: float, shipping_cost: float) -> dict:
    """Calcule le coût total, incluant le produit et les frais de port."""
    total = product_price + shipping_cost

```

```

    return {
        "status": "success",
        "subtotal": product_price,
        "shipping": shipping_cost,
        "total": total
    }

# Agent doté de plusieurs outils
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="""
    Lorsqu'un utilisateur demande le coût total :
    1. Commence par vérifier l'inventaire avec check_inventory.
    2. Si le produit est en stock, obtiens son prix avec get_product_price.
    3. Calcule les frais de port avec calculate_shipping.
    4. Utilise calculate_total pour obtenir le prix final.

    Présente clairement le détail des informations au client.
    """,
    tools=[check_inventory, get_product_price, calculate_shipping, calculate_total]
)

```

Comment l'agent orchestre l'exécution :

Utilisateur : "Quel est le coût total du produit PROD001 expédié au Canada ?"

Raisonnement de l'agent :

1. L'agent appelle check_inventory("PROD001") → in_stock: True
2. L'agent appelle get_product_price("PROD001") → price: \$99.99
3. L'agent appelle calculate_shipping(weight, "Canada") → shipping: \$25.50
4. L'agent appelle calculate_total(99.99, 25.50) → total: \$125.49

L'agent répond : "Le produit PROD001 est disponible. Il coûte 99,99 \$ et les frais de port pour le Canada s'élèvent à 25,50 \$, pour un total de 125,49 \$."

Points essentiels :

- **Appels séquentiels** : l'agent appelle les outils dans un ordre logique.
- **Enchaînement des résultats** : la sortie d'un outil sert de contexte au suivant.
- **Orchestration automatique** : l'agent décide des outils à utiliser et quand.
- **Synthèse naturelle** : l'agent combine tous les résultats dans une réponse cohérente.

Exemple pratique

Créer un agent de voyage avec des outils personnalisés

Objectif : Créer un agent de voyage qui aide les utilisateurs à planifier des escapades en recherchant des vols et des hôtels, et en calculant des budgets.

Étape 1 : Créer le projet

```
adk create travel_agent
cd travel_agent
```

Fonction :

- Crée un nouveau répertoire de projet ADK
 - Établit une structure d'agent de base
 - Crée `agent.py` avec le code par défaut
-

Étape 2 : Écrire l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent de voyage avec outils de fonction personnalisés
Démontre le fonctionnement conjoint de plusieurs outils personnalisés.

Référence : https://google.github.io/adk-docs/tools-custom/function-tools/
"""

from google.adk.agents import LlmAgent

# Outil 1 : recherche de vols
def search_flights(destination: str, departure_date: str) -> dict:
    """Recherche les vols disponibles vers une destination à une date précise.

    Utilise cet outil lorsqu'un client souhaite connaître les options de vol.

    Args:
        destination (str) : la ville de destination (par exemple, "Paris", "Tokyo").
        departure_date (str) : date de départ au format AAAA-MM-JJ.

    Renvoie :
        dict : résultats de la recherche de vols.
            En cas de succès : {'status': 'success', 'flights': [...], 'count': N}
            En cas d'erreur : {'status': 'error', 'error_message': 'explanation'}
    """
    # Données de vol simulées
    available_flights = {
        "paris": [
            {"flight_number": "AF123", "price_usd": 450, "duration_hours": 8},
            {"flight_number": "BA456", "price_usd": 480, "duration_hours": 7.5},
        ],
        "tokyo": [
            {"flight_number": "JL789", "price_usd": 850, "duration_hours": 13},
            {"flight_number": "ANA101", "price_usd": 820, "duration_hours": 12.5},
        ],
    }
}
```

```

    dest_key = destination.lower()
    if dest_key not in available_flights:
        return {
            "status": "error",
            "error_message": f"Aucun vol trouvé pour {destination}. Essayez Paris ou
Tokyo."
        }

    return {
        "status": "success",
        "destination": destination,
        "departure_date": departure_date,
        "flights": available_flights[dest_key],
        "count": len(available_flights[dest_key])
    }

```

Outil 2 : recherche d'hôtels

```
def search_hotels(city: str, check_in_date: str) -> dict:
```

"""Recherche des hôtels disponibles dans une ville pour une date d'arrivée spécifique.

Utilise cet outil lorsqu'un client a besoin d'un hébergement.

Args:

city (str) : le nom de la ville (par exemple, "Paris", "Tokyo").

check_in_date (str) : date d'arrivée au format AAAA-MM-JJ.

Renvoie :

dict : résultats de la recherche d'hôtels.

En cas de succès : {'status': 'success', 'hotels': [...], 'count': N}

En cas d'erreur : {'status': 'error', 'error_message': 'explanation'}

"""

Données d'hôtels simulées

```

available_hotels = {
    "paris": [
        {"name": "Hotel Eiffel", "price_per_night_usd": 150, "rating": 4.5},
        {"name": "Louvre Inn", "price_per_night_usd": 120, "rating": 4.2},
    ],
    "tokyo": [
        {"name": "Shibuya Grand", "price_per_night_usd": 180, "rating": 4.7},
        {"name": "Tokyo Bay Hotel", "price_per_night_usd": 140, "rating": 4.3},
    ],
}

```

```
city_key = city.lower()
```

```
if city_key not in available_hotels:
```

```
    return {
```

```
        "status": "error",
```

```
        "error_message": f"Aucun hôtel trouvé à {city}. Essayez Paris ou Tokyo."
    }
```

```
return {
```

```
    "status": "success",
```

```

        "city": city,
        "check_in_date": check_in_date,
        "hotels": available_hotels[city_key],
        "count": len(available_hotels[city_key])
    }

# Outil 3 : calculer le budget du voyage
def calculate_trip_budget(flight_price: float, hotel_price: float, num_nights: int) -> dict:
    """Calcule le budget total du voyage, vols et hébergement compris.

    Utilise ceci après avoir trouvé les prix des vols et des hôtels pour donner au
    client une estimation totale.

    Args:
        flight_price (float) : coût du vol aller-retour en USD.
        hotel_price (float) : coût de l'hôtel par nuit en USD.
        num_nights (int) : nombre de nuits sur place.

    Renvoie :
        dict : répartition du budget.
        Renvoie toujours : {'status': 'success', 'total_usd': X, 'breakdown':
{...}}
    """
    hotel_total = hotel_price * num_nights
    total = flight_price + hotel_total

    return {
        "status": "success",
        "total_usd": round(total, 2),
        "breakdown": {
            "flight_cost": flight_price,
            "hotel_cost_per_night": hotel_price,
            "num_nights": num_nights,
            "hotel_total": round(hotel_total, 2)
        }
    }

# Créer un agent de voyage avec les trois outils
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='travel_agent',
    description='Aide les utilisateurs à planifier leurs voyages en trouvant des vols
et des hôtels.',
    instruction="""
    Tu es un agent de voyage serviable.

    Tes capacités :
    - Rechercher des vols à l'aide de search_flights(destination, departure_date)
    - Rechercher des hôtels à l'aide de search_hotels(city, check_in_date)
    - Calculer des budgets de voyage à l'aide de calculate_trip_budget(flight_price,
hotel_price, num_nights)

    Lorsque tu aides les utilisateurs, procède comme suit :

```

```

1. S'ils posent des questions sur les vols, utilise search_flights.
2. S'ils posent des questions sur les hôtels, utilise search_hotels.
3. S'ils souhaitent une estimation complète du voyage, utilise les deux outils de
recherche, puis calculate_trip_budget.
4. Présente toujours clairement les différentes options et leurs prix.
5. Si un outil renvoie une erreur, présente tes excuses et suggère d'autres
destinations disponibles (Paris ou Tokyo).

    Sois aimable et aide les utilisateurs à planifier leur voyage idéal !
    """
    tools=[search_flights, search_hotels, calculate_trip_budget]
)

```

Tutoriel du code :

Lignes 10 à 50 : outil search_flights

- Nom descriptif : `search_flights` (modèle verbe-nom)
- Indications de type : `str, str → dict`
- Docstring complète expliquant quoi, quand, les arguments et les valeurs de retour
- Renvoie un dictionnaire structuré avec la clé `status`
- Gestion des exceptions pour les destinations non disponibles

Lignes 52 à 91 : outil search_hotels

- Modèle semblable à `search_flights`
- Structure de données différente (hôtels vs vols)
- Gestion cohérente des exceptions avec `status` et `error_message`

Lignes 93 à 116 : outil calculate_trip_budget

- Les sorties d'autres outils servent d'entrées (`flight_price`, `hotel_price`).
- Le calcul est effectué.
- Le détail du calcul est renvoyé.
- L'opération réussit toujours (`status: success`).

Lignes 118 à 142 : agent doté de plusieurs outils

- Les trois outils de la liste `tools`
- Les instructions indiquent quand utiliser chaque outil
- Explique comment gérer les exceptions
- Guide d'utilisation séquentielle pour les requêtes complexes

Étape 3 : Exécuter et tester

`adk web`

Accédez à <http://localhost:8000> dans votre navigateur.

Étape 4 : Scénarios de test

Test 1 : Recherche de vol

Vous : Je souhaite prendre un vol pour Paris le 15/12/2025.

Comportement attendu :

- L'agent appelle `search_flights(destination="Paris", departure_date="2025-12-15")`.
- L'outil renvoie deux options de vol.
- L'agent présente ces options avec les numéros de vol, leur prix et leur durée.

Points à noter :

- L'agent a automatiquement extrait la destination et la date à partir du langage naturel.
- L'outil a renvoyé des données structurées avec plusieurs options de vol.
- L'agent a présenté les résultats à l'utilisateur de manière naturelle.

Test 2 : Planification complète de voyage

Vous : Planifie un voyage de trois nuits à Tokyo à partir du 20/12/2025.

Comportement attendu :

1. L'agent appelle `search_flights(destination="Tokyo", departure_date="2025-12-20")`.
2. L'agent appelle `search_hotels(city="Tokyo", check_in_date="2025-12-20")`.
3. L'agent sélectionne un vol et un hôtel raisonnables parmi les options disponibles.
4. L'agent appelle `calculate_trip_budget(flight_price, hotel_price, 3)`.
5. L'agent présente un itinéraire complet avec le détail du coût total.

Points à noter :

- L'agent a orchestré trois appels d'outils différents.
- Les outils ont œuvré ensemble de manière séquentielle.
- La réponse finale synthétise toutes les informations.
- Les données structurées sont présentées en langage naturel.

Test 3 : Gestion des exceptions

Vous : Trouve des hôtels à Londres.

Comportement attendu :

- L'agent appelle `search_hotels(city="London", check_in_date=...)`.
- L'outil renvoie une erreur : `{"status": "error", "error_message": "Aucun hôtel trouvé à Londres. Essayez Paris ou Tokyo."}`
- L'agent s'excuse poliment.
- L'agent suggère d'autres destinations disponibles.

Points à noter :

- La gestion des exceptions est optimale.
 - L'agent n'hallucine pas des données d'hôtels pour les villes non disponibles.
 - Les destinations disponibles sont clairement indiquées.
 - Le ton reste professionnel malgré les erreurs.
-

Test 4 : Calcul de budget uniquement

Vous : Quel est le coût total pour un vol à 450 \$ et un hôtel à 150 \$/nuit pendant quatre nuits ?

Comportement attendu :

- L'agent appelle `calculate_trip_budget(flight_price=450, hotel_price=150, num_nights=4)`.
- L'outil renvoie le détail du calcul.
- L'agent répond : un vol à 450 \$ + un hôtel à 600 \$ (4 nuits × 150 \$) = un total de 1 050 \$.

Points à noter :

- L'agent a sélectionné l'outil approprié pour la tâche.
 - L'agent n'a pas appelé d'outils de recherche inutilement.
 - L'outil se concentre sur une tâche précise : le calcul.
 - Le détail du calcul aide l'utilisateur à comprendre le total.
-

Points à retenir

Principes de base des outils de fonction :

- **Noms de fonctions descriptifs** : utilisez la structure verbe-nom (`get_`, `calculate_`, `search_*`).
- **Indications de type requises** : ADK les utilise pour générer le schéma du LLM.
- **Docstrings complètes** : expliquez le fonctionnement de l'outil, son utilisation, ses arguments et ses valeurs de retour.

- **Dictionnaires renvoyés** : incluez toujours la clé **status** pour la compréhension du LLM.

Bonnes pratiques :

```
def tool_name(param: type) -> dict:
    """Résumé clair en une ligne.

    Contexte supplémentaire sur les situations dans lesquelles utiliser cet outil.

    Args:
        param (type) : description du paramètre.

    Renvoie :
        dict : description de la valeur de retour.
            En cas de succès : {'status': 'success', 'data': value}
            En cas d'erreur : {'status': 'error', 'error_message': 'explanation'}
    """
    # Valider les entrées
    if error_condition:
        return {"status": "error", "error_message": "Explication claire"}

    # Effectuer l'opération
    return {"status": "success", "result_key": computed_value}
```

Outils multiples :

- **Lister toutes les fonctions** : `tools=[tool1, tool2, tool3]`
- **L'agent sélectionne automatiquement** : en fonction du contexte et des docstrings
- **Utilisation séquentielle** : les outils peuvent s'appuyer sur les résultats des autres outils
- **Référence dans les instructions** : indique au LLM quand utiliser chaque outil

Génération de schémas :

ADK crée automatiquement le schéma de l'outil à partir des éléments suivants :

1. Nom de la fonction
2. Noms des paramètres et indications de type
3. Docstring
4. Type renvoyé

Quand utiliser des outils personnalisés ?

- ☒ Calculs spécifiques à l'entreprise (frais de port, remises, taxes)
- ☒ Recherches dans des bases de données propriétaires (commandes, inventaire, clients)
- ☒ Intégrations d'API personnalisées (vos services internes)
- ☒ Logique spécifique au domaine (systèmes de réservation, règles de tarification)
- ☒ Toute tâche propre à votre application

Différences majeures :

Aspect	Outils intégrés	Outils de fonction personnalisés
Configuration	Importer et utiliser	Écrire l'implémentation de la fonction
Maintenance	Maintenance par l'équipe ADK	Maintenance par vous
Personnalisation	Limitée aux fonctionnalités de l'outil	Contrôle complet
Utilisation	Capacités courantes	Logique spécifique à l'entreprise